

Dokumentacja testów

Załącznik B do raportu końcowego

Zespół PlayLink

19 czerwca 2026

Treść niniejszego dokumentu jest tożsama z rozdziałem „Testowanie rozwiązania” raportu końcowego.

1 Testowanie rozwiązania

PlayLink jest pokryty obszernym, zautomatyzowanym zestawem testów: około **133 testy** po stronie backendu (pytest) oraz około **60 testów** po stronie frontendu (Vitest). Pod względem *różnorodności* wyróżniono **sześć różnych rodzajów testów** (powyżej wymaganego minimum pięciu), zestawionych w tab. 1. W podrozdziale 1.2 opisano ponadto **dziesięć reprezentatywnych przypadków testowych** (po kilka na rodzaj) wraz z celem, metodą i wynikiem.

Uwaga terminologiczna (różnorodność vs. liczba). Kryterium oceny dotyczy *rodzajów* (typów) testów — wymaganych jest 5 różnych. Liczba opisanych przypadków (10) to inna oś: są to konkretne testy rozłożone na owe rodzaje (np. 2 na rodzaj). Oba wymagania są zatem spełnione jednocześnie: 6 rodzajów oraz 10 szczegółowo udokumentowanych przypadków.

Rodzaj testów	Narzędzie	Zakres	Liczba
Jednostkowe	pytest, Vitest	Funkcje pomocnicze, walidacja, store’y, metryka Haversine	~25
Integracyjne / API	FastAPI TestClient	Endpointy REST przez całą aplikację (SQLite in-memory)	~54
WebSocket / real-time	TestClient WS	Rozgłaszanie ramek czatu i zdarzeń (34 połączenia WS)	~45
Bezpieczeństwa	pytest + eth_account	Replay, wygasły nonce, fałszywy admin, podrobione podpisy	~10
Migracji (smoke)	pytest + Alembic	Migracja do <code>head</code> , weryfikacja schematu; w CI round-trip na Postgresie	1 (+CI)
Kontraktowe (frontend)	Vitest	Server-loadery i akcje SvelteKit (+ <code>page.server.ts</code>)	~20

Tabela 1: Sześć rodzajów testów w projekcie PlayLink.

1.1 Charakterystyka rodzajów testów

Testy jednostkowe. Weryfikują izolowaną logikę bez bazy i sieci: parsowanie listy adresów administratorów, funkcję `hash_nonce`, walidację nazw użytkownika, a we frontendzie — podpisywanie, store’y oraz obliczanie odległości lobby (Haversine). Narzędzia: `pytest`, `Vitest` + `jsdom`.

Testy integracyjne / API. Uruchamiają żądania HTTP przez całą aplikację FastAPI z użyciem `TestClient` i bazy SQLite in-memory (fixture nadpisuje `get_session`). Pokrywają uwierzytelnianie, pełny cykl życia pokoju, panel administratora i akcję kick — sprawdzając kody statusu oraz stan w bazie.

Testy WebSocket / realtime. Wykorzystują `TestClient.websocket_connect` (34 połączenia WS) do weryfikacji rozgłaszania ramek: `event-update`, `rsvp_update`, `roster_update`, `room_closed`, `member_kicked`.

Testy bezpieczeństwa. Najważniejszy rodzaj dla profilu projektu — scenariusze ataków (replay, wygasły nonce, eskalacja uprawnień, podrobione podpisy EIP-191), z realnym podpisywaniem kluczem (`eth_account`).

Testy migracji (smoke). Aplikują pełny łańcuch Alembic (`alembic upgrade head`) i sprawdzają obecność wszystkich ośmiu tabel; w CI powtarzane na realnym PostgreSQL z round-tripem downgrade/upgrade i `alembic check`.

Testy kontraktowe frontendu. Importują server-loadery i akcje SvelteKit z zamockowanymi `fetch/cookies`, weryfikując przekierowania, strażników autoryzacji i walidację formularzy.

1.2 Katalog wybranych przypadków testowych

Poniżej opisano dziesięć reprezentatywnych przypadków testowych w jednolitym formacie (rodzaj, cel, metoda/narzędzie, dane/kroki, oczekiwany wynik, rezultat). Wszystkie przechodzą pomyślnie w pipeline CI; zespół może dołączyć do każdego zrzut ekranu z wynikiem (np. raport pytest/Vitest lub log CI).

TC-01: Parsowanie adresów administratorów

<i>Rodzaj</i>	Jednostkowy
<i>Cel</i>	Sprawdzić poprawne wczytanie i normalizację listy adresów z ADMIN_ADDRESSES.
<i>Metoda / narzędzie</i>	pytest (test_helpers.py, _parse_admin_addresses).
<i>Dane / kroki</i>	Wejście " 0xABC , 0xdef " (spacje, różna wielkość liter).
<i>Oczekiwany wynik</i>	Zbiór znormalizowanych adresów małymi literami: {0xabc, 0xdef}.
<i>Rezultat</i>	PASS

TC-02: Hashowanie wartości nonce

<i>Rodzaj</i>	Jednostkowy
<i>Cel</i>	Zapewnić, że nonce jest deterministycznie hashowany (SHA-256) przed zapisem.
<i>Metoda / narzędzie</i>	pytest (hash_nonce).
<i>Dane / kroki</i>	Ten sam nonce hashowany dwukrotnie oraz dwa różne nonce.
<i>Oczekiwany wynik</i>	Identyczny skrót dla identycznego wejścia; różne skróty dla różnych wejść.
<i>Rezultat</i>	PASS

TC-03: Obliczanie odległości lobby (Haversine)

<i>Rodzaj</i>	Jednostkowy (frontend)
<i>Cel</i>	Zweryfikować dobór pingu na podstawie odległości geograficznej lobby.
<i>Metoda / narzędzie</i>	Vitest (lobbyLocations.test.ts).
<i>Dane / kroki</i>	Pary współrzędnych o znanej odległości.
<i>Oczekiwany wynik</i>	Odległość w granicach tolerancji; właściwy przedział pingu.
<i>Rezultat</i>	PASS

TC-04: Pełny przepływ logowania (challenge-response)

<i>Rodzaj</i>	Integracyjny / API
<i>Cel</i>	Zweryfikować ścieżkę request-nonce → verify → wydanie JWT.
<i>Metoda / narzędzie</i>	FastAPI TestClient (test_auth.py), podpis eth_account.
<i>Dane / kroki</i>	POST /auth/request-nonce, podpis nonce, POST /auth/verify.
<i>Oczekiwany wynik</i>	HTTP 200 oraz poprawny token JWT i nazwa użytkownika w odpowiedzi.
<i>Rezultat</i>	PASS

TC-05: Cykl życia pokoju i limit aktywnych pokoi

<i>Rodzaj</i>	Integracyjny / API
<i>Cel</i>	Sprawdzić tworzenie, dołączanie, opuszczanie oraz limit 3 aktywnych pokoi.
<i>Metoda / narzędzie</i>	FastAPI TestClient (test_rooms_lifecycle.py).
<i>Dane / kroki</i>	Sekwencja create/join/leave; próba utworzenia 4. pokoju.
<i>Oczekiwany wynik</i>	Operacje 1–3 kończą się sukcesem; 4. pokój odrzucony błędem walidacji.
<i>Rezultat</i>	PASS

TC-06: Rozgłaszanie wiadomości i ramek w czacie

<i>Rodzaj</i>	WebSocket / realtime
<i>Cel</i>	Potwierdzić, że klienci otrzymują aktualizacje w czasie rzeczywistym.
<i>Metoda / narzędzie</i>	TestClient.websocket_connect (test_chat.py, test_room_events.py).
<i>Dane / kroki</i>	Dwa połączenia WS w jednym pokoju; wysłanie wiadomości i zmiana RSVP.
<i>Oczekiwany wynik</i>	Obaj klienci otrzymują ramki message, rsvp-update, roster-update.
<i>Rezultat</i>	PASS

TC-07: Odrzucenie ataku powtórzeniowego (replay)

<i>Rodzaj</i>	Bezpieczeństwa
<i>Cel</i>	Uniemożliwić ponowne użycie tego samego nonce.
<i>Metoda / narzędzie</i>	pytest (<code>test_verify_signature_rejects_replayed_nonce</code>).
<i>Dane / kroki</i>	Poprawna weryfikacja, a następnie ponowna z tym samym nonce/podpisem.
<i>Oczekiwany wynik</i>	Pierwsza weryfikacja: 200; druga: odrzucenie (nonce oznaczony jako użyty).
<i>Rezultat</i>	PASS

TC-08: Brak eskalacji uprawnień administratora

<i>Rodzaj</i>	Bezpieczeństwa
<i>Cel</i>	Sfałszowane roszczenie o rolę admina nie może dawać dostępu.
<i>Metoda / narzędzie</i>	pytest (<code>test_forged_admin_claim_does_not_grant_admin_access</code>).
<i>Dane / kroki</i>	Token/żądanie konta spoza ADMIN_ADDRESSES próbuje akcji administracyjnej.
<i>Oczekiwany wynik</i>	HTTP 403; operacja administracyjna nie zostaje wykonana.
<i>Rezultat</i>	PASS

TC-09: Migracja schematu bazy (smoke)

<i>Rodzaj</i>	Migracji
<i>Cel</i>	Zapewnić, że pełny łańcuch migracji buduje poprawny schemat.
<i>Metoda / narzędzie</i>	pytest + Alembic (<code>test_migrations.py</code>); w CI na PostgreSQL.
<i>Dane / kroki</i>	<code>alembic upgrade head</code> na świeżej bazie; inspekcja schematu.
<i>Oczekiwany wynik</i>	Obecnych wszystkie 8 tabel i kluczowe kolumny; brak dryfu (<code>alembic check</code>).
<i>Rezultat</i>	PASS

TC-10: Strażnik autoryzacji trasy (frontend)

<i>Rodzaj</i>	Kontraktowy (frontend)
<i>Cel</i>	Niezałogowany użytkownik nie ma dostępu do tras chronionych.
<i>Metoda / narzędzie</i>	Vitest (<code>rooms.name.page.server.test.ts</code>), zamockowane <code>cookies/fetch</code> .
<i>Dane / kroki</i>	Wywołanie <code>load</code> bez ważnej sesji.
<i>Oczekiwany wynik</i>	Przekierowanie HTTP 303 do <code>/rooms</code> ; brak ujawnienia danych chronionych.
<i>Rezultat</i>	PASS

1.3 Dokumentacja procesu i automatyzacja (CI)

Proces testowy jest opisany w repozytorium (`docs/operations/testing.md` — tabele pokrycia per plik, opis fixture'ów i kanałów WS). Konfiguracja: `pyproject.toml` (`[tool.pytest.ini_options]`, `pytest-cov`), `vite.config.js` (Vitest, `jsdom`, `coverage-v8`). Pipeline GitHub Actions (`.github/workflows/cicd.yml`) uruchamia zadania: `backend-lint`, `backend-test` (z bramką `-cov-fail-under=80`), `backend-migrations`, `frontend-lint`, `frontend-test` (z pokryciem) oraz `build-test`. Zielony wynik wszystkich testów jest warunkiem wdrożenia (`deploy`) oraz scalenia PR do `main`, co stanowi ciągłą, weryfikowalną dokumentację wyników testowania.